go-agent 核心架构文档

系统性阅读与微调测试参考文档

Protocol-Lattice/go-agent

2026 年 3 月 3 日

目录

0.1 项目概览 3
0.1.1 基本信息 3
0.1.2 项目结构 3
0.2 核心架构 4
0.2.1 架构图 4
0.2.2 核心组件关系 5
0.3 Agent 核心实现 5
0.3.1 文件位置 5
0.3.2 核心结构体 5
0.3.3 核心方法 6
0.3.4 工具调用流程 7
0.3.5 SubAgent 委托 7
0.4 ADK - Agent 开发工具包 7
0.4.1 文件位置 7
0.4.2 核心结构体 8
0.4.3 创建流程 8
0.4.4 模块系统 9
0.4.5 提供者模式 9
0.5 记忆系统 10
0.5.1 文件位置 10
0.5.2 架构层次 10
0.5.3 核心概念 10
0.5.4 记忆引擎 12
0.5.5 存储适配器 13
0.5.6 Embedding 模型 13
0.6 模型适配器 14
0.6.1 文件位置 14
0.6.2 Agent 接口 14
0.6.3 支持的提供商 15
0.6.4 缓存模型 16

0.7 SubAgent 系统 16
0.7.1 文件位置 16
0.7.2 SubAgent 接口 16
0.7.3 预置 SubAgent 17
0.7.4 自定义 SubAgent 17
0.7.5 SubAgent 作为工具 18
0.8 工具系统 18
0.8.1 文件位置 18
0.8.2 Tool 接口 18
0.8.3 ToolSpec 19
0.8.4 示例：计算器工具 19
0.8.5 UTCP 工具调用 19
0.9 安全护栏 20
0.9.1 文件位置 20
0.9.2 OutputGuardrails 20
0.9.3 预置策略 20
0.9.4 使用示例 21
0.10 微调测试指南 22
0.10.1 测试 1: 修改提示词 22
0.10.2 测试 2: 修改记忆配置 22
0.10.3 测试 3: 添加 SubAgent 23
0.10.4 测试 4: 添加工具 23
0.10.5 测试 5: 修改安全护栏 24
0.11 附录 25
0.11.1 环境变量参考 25
0.11.2 常用命令 25

1. 项目概览
2. 核心架构
3. Agent 核心实现
4. ADK - Agent 开发工具包
5. 记忆系统
6. 模型适配器
7. SubAgent 系统
8. 工具系统
9. 安全护栏
10. 微调测试指南

0.1 项目概览

0.1.1 基本信息

- **名称**: Lattice (go-agent)
- **语言**: Go 1.25+
- **用途**: 构建 AI 代理的开发框架
- **核心特性**:
  - 模块化架构 - 通过声明式配置组合代理
  - 多代理支持 - 专家代理通过共享目录协调
  - 丰富工具系统 - 实现 `Tool` 接口即可使用
  - 智能记忆 - RAG 驱动，重要性评分，MMR 检索
  - 模型无关 - Gemini, Anthropic, Ollama 适配器
  - UTCP 支持 - 通用工具调用协议
  - 高性能 - LRU 缓存，预分配缓冲区

0.1.2 项目结构

```
go-agent/
  cmd/                    # 命令行应用和示例
     app/               # 主应用程序
     example/           # 示例项目
        5ways_demo/           # 5 种通信方式演示
        agent_as_tool/        # Agent 作为工具
        autonomous_agent/     # 自主代理 CLI
        guardrails/           # 安全护栏
        hybrid_memory_test/   # 混合记忆测试
        multi_agent_memory/   # 多代理共享记忆
        ...
     codemode/        # CodeMode 独立命令

  src/                    # 核心源代码
     adk/               # Agent Development Kit
     memory/            # 记忆系统
     models/            # LLM 提供商适配器
     subagents/         # 预构建专家代理
     tools/             # 工具系统
     swarm/             # 多代理协调
     cache/             # 缓存优化
```

```
    helpers/           # 辅助函数

docs/                   # 文档
agent.go                # Agent 核心实现 (1819 行)
agent_tool.go           # Agent 作为工具 (318 行)
agent_orchestrators.go  # 编排器
README.md               # 项目说明
```

0.2 核心架构

0.2.1 架构图

```
                    User Application



             Agent Development Kit (ADK)

    Modules        Providers       Runtime Glue



  Agent          Memory         Tools
  Core           System         System

- Model        - Engine       - Catalog
- Session      - Session      - SubAgent
- UTCP         - Store        - UTCP
- CodeMode     - Embed        - Chain
```

0.2.2 核心组件关系

1. **ADK** (AgentDevelopmentKit)
   - 依赖注入容器
   - 模块引导和注册
   - 提供 Model/Memory/Tool/SubAgent
2. **Agent**
   - 核心编排器
   - 调用模型、记忆、工具
   - 管理 SubAgent
3. **Memory**
   - 会话记忆（短期）
   - 共享空间（长期）
   - 向量存储（PostgreSQL/Qdrant）
4. **Tools**
   - 工具目录
   - SubAgent 适配器
   - UTCP 工具调用

0.3 Agent 核心实现

0.3.1 文件位置

`agent.go` (1819 行)

0.3.2 核心结构体

```go
type Agent struct {
    // LLM 模型
    model        models.Agent

    // 记忆系统
    memory       *memory.SessionMemory
    systemPrompt string
    contextLimit int

    // 工具目录
```

```
    toolCatalog       ToolCatalog
    subAgentDirectory SubAgentDirectory
    UTCPClient        utcp.UtcpClientInterface

    // 共享记忆
    Shared    *memory.SharedSession

    // CodeMode 工作流
    CodeMode  *codemode.CodeModeUTCP
    CodeChain *chain.UtcpChainClient

    // 安全护栏
    Guardrails       *OutputGuardrails
    AllowUnsafeTools bool

    // 互斥锁
    mu sync.Mutex
}
```

0.3.3 核心方法

0.3.3.1 1. Generate - 单次生成

```
func (a *Agent) Generate(ctx context.Context, prompt string) (any, error)
```

流程: 1. 加载会话记忆 2. 构建提示词（系统提示 + 用户输入 + 记忆）3. 调用 LLM 模型 4. 解析工具调用（如果有）5. 执行工具 6. 返回结果

0.3.3.2 2. GenerateWithFiles - 带文件上下文

```
func (a *Agent) GenerateWithFiles(ctx context.Context, prompt string, files []models.File) (any, e
```

特点: - 支持多模态输入（文本 + 文件）- 自动处理文件编码 - 内联文本文件，引用非文本文件

0.3.3.3 3. GenerateStream - 流式生成

```
func (a *Agent) GenerateStream(ctx context.Context, prompt string) (<-chan models.StreamChunk, err
```

特点: - 支持流式输出 - 实时显示生成过程 - 可中途取消

0.3.4 工具调用流程

```
// 1. 检查是否需要调用工具
if containsToolCall(response) {
    // 2. 解析工具调用
    toolCalls := parseToolCalls(response)

    // 3. 执行工具
    for _, call := range toolCalls {
        result := a.InvokeTool(ctx, call)

        // 4. 将结果添加到对话
        conversation = append(conversation, result)
    }

    // 5. 再次调用模型生成最终回复
    finalResponse := a.Generate(ctx, conversation)
}
```

0.3.5 SubAgent 委托

```
// 1. 检查是否委托给 SubAgent
if subAgent, ok := a.subAgentDirectory.Get(toolName); ok {
    // 2. 调用 SubAgent
    result := subAgent.Run(ctx, instruction)

    // 3. 返回结果
    return result
}
```

0.4 ADK - Agent 开发工具包

0.4.1 文件位置

`src/adk/kit.go` (467 行)

0.4.2 核心结构体

```go
type AgentDevelopmentKit struct {
    mu sync.RWMutex

    // 模块系统
    modules       []Module
    bootstrapped bool

    // 提供者
    modelProvider    ModelProvider
    memoryProvider   MemoryProvider
    memoryInstance   *memory.SessionMemory
    sharedFactory    SharedSessionFactory
    toolProviders    []ToolProvider
    subAgentProvider []SubAgentProvider

    // 默认配置
    defaultSystemPrompt string
    defaultContextLimit int

    // Agent 选项
    agentOptions []AgentOption

    // UTCP 客户端
    UTCP         utcp.UtcpClientInterface
    CodeMode     *codemode.CodeModeUTCP
    ChainMode    *chain.UtcpChainClient
}
```

0.4.3 创建流程

```go
// 1. 创建 ADK
adk, err := adk.New(ctx,
    // 2. 配置 SubAgent
    adk.WithSubAgents(subagents.NewResearcher(researcherModel)),

    // 3. 配置模块
    adk.WithModules(
```

```
        // 模型模块
        adkmodules.NewModelModule("gemini-model", func(_ context.Context) (models.Agent, error) {
            return models.NewGeminiLLM(ctx, "gemini-2.5-pro", "Swarm orchestration:")
        }),

        // 记忆模块
        adkmodules.InQdrantMemory(100000, *qdrantURL, *qdrantCollection, memory.AutoEmbedder(), &me
    ),
)

// 4. Bootstrap 引导（执行所有模块）
if err := kit.Bootstrap(ctx); err != nil {
    log.Fatal(err)
}
```

0.4.4 模块系统

0.4.4.1 Module 接口

```
type Module interface {
    Execute(ctx context.Context, kit *AgentDevelopmentKit) error
}
```

0.4.4.2 预置模块

1. **ModelModule** - 模型提供者
2. **MemoryModule** - 记忆系统
3. **ToolModule** - 工具提供者
4. **SubAgentModule** - SubAgent 提供者

0.4.5 提供者模式

```
// ModelProvider - 模型提供者
type ModelProvider interface {
    ProvideModel(ctx context.Context) (models.Agent, error)
}

// MemoryProvider - 记忆提供者
type MemoryProvider interface {
```

```go
    ProvideMemory(ctx context.Context) (*memory.SessionMemory, error)
}

// ToolProvider - 工具提供者
type ToolProvider interface {
    ProvideTools(ctx context.Context) ([]Tool, error)
}
```

0.5 记忆系统

0.5.1 文件位置

```
src/memory/
```

0.5.2 架构层次

```
memory/
  memory.go              # 类型定义和工厂函数
  engine/
      engine.go          # 记忆引擎（重要性评分、检索）
  session/
      session.go         # 会话记忆
      shared_session.go  # 共享会话
  store/
      postgres.go        # PostgreSQL 存储
      qdrant.go          # Qdrant 向量存储
      inmemory.go        # 内存存储
  embed/
       embed.go           # Embedding 模型
```

0.5.3 核心概念

0.5.3.1 1. MemoryBank - 记忆库

```go
type MemoryBank interface {
    // 存储记忆
```

```
    Store(ctx context.Context, records []*model.MemoryRecord) error

    // 检索记忆
    Retrieve(ctx context.Context, query string, k int) ([]*model.MemoryRecord, error)

    // 删除记忆
    Delete(ctx context.Context, ids []string) error
}
```

0.5.3.2 2. SessionMemory - 会话记忆

```
type SessionMemory struct {
    bank    MemoryBank
    session string  // 会话 ID
    limit   int     // 检索数量
    Spaces  SpaceRegistry  // 共享空间注册表
}
```

使用示例:

```
// 创建会话记忆
sessionMem := memory.NewSessionMemory(bank, 16)

// 存储记忆
sessionMem.StoreLong(ctx, "My name is Alice")

// 检索记忆
records, _ := sessionMem.Retrieve(ctx, "What is my name?", 5)
```

0.5.3.3 3. SharedSession - 共享会话

```
type SharedSession struct {
    inner *SessionMemory
    agent string  // Agent 名称
    space string  // 共享空间名称
}
```

使用示例:

```
// 创建共享会话
shared := session.NewSharedSession(sessionMem, "researcher", "team:project-x")
```

```go
// 存储到共享空间
shared.StoreLongTo(ctx, "team:project-x", "Project requirement: Build REST API", nil)

// 从共享空间检索
records, _ := shared.Retrieve(ctx, "project requirement", 5)
```

0.5.3.4 4. SpaceRegistry - 空间注册表

```go
type SpaceRegistry interface {
    // 授予访问权限
    Grant(space, agent string, role SpaceRole, expiresAt int64)

    // 撤销访问权限
    Revoke(space, agent string)

    // 检查权限
    CanAccess(space, agent string, role SpaceRole) bool
}
```

权限级别: - SpaceRoleReader - 只读 - SpaceRoleWriter - 读写 - SpaceRoleAdmin - 管理员

0.5.4 记忆引擎

0.5.4.1 重要性评分

```go
type ScoreWeights struct {
    RecencyWeight    float64  // 新近度权重 (默认 0.3)
    ImportanceWeight float64  // 重要性权重 (默认 0.5)
    RelevanceWeight  float64  // 相关性权重 (默认 0.2)
}
```

评分公式:

```
final_score = recency_score * recency_weight +
              importance_score * importance_weight +
              relevance_score * relevance_weight
```

0.5.4.2 MMR 检索 (Maximal Marginal Relevance)

```
// 平衡相关性和多样性
func mmr(query string, candidates []*MemoryRecord, lambda float64, k int) []*MemoryRecord
```

参数: - `lambda`: 相关性权重 (0-1, 默认 0.5) - `k`: 返回数量

0.5.5 存储适配器

0.5.5.1 PostgreSQL 存储

```
// 创建存储
pgStore, err := store.NewPostgresStore(ctx, "postgres://admin:admin@localhost:5432/ragdb?sslmode=d

// 创建记忆库
bank := memory.NewMemoryBankWithStore(pgStore)
```

数据库表: - `memory_nodes` - 记忆节点 - `memory_edges` - 记忆关系 - `memory_sessions` - 会话 - `memory_spaces` - 共享空间

0.5.5.2 Qdrant 存储

```
// 创建存储
qdrantStore, err := store.NewQdrantStore(ctx, "http://localhost:6333", "adk_memories")

// 配置向量维度
request := store.CreateCollectionRequest{
    CollectionName: "adk_memories",
    VectorSize:     768,  // 根据 Embedding 模型调整
    Distance:       store.Cosine,
}
```

0.5.6 Embedding 模型

0.5.6.1 自动选择

```
// 根据环境变量自动选择 Embedding 模型
embedder := memory.AutoEmbedder()

// 环境变量优先级:
```

```
// 1. OPENAI_API_KEY → OpenAI Embedding
// 2. VERTEX_AI_PROJECT_ID → VertexAI Embedding
// 3. OLLAMA_HOST → Ollama Embedding
// 4. 默认 → DummyEmbedder (零向量)
```

0.5.6.2 手动指定

```
// OpenAI
embedder, _ := embed.NewOpenAIEmbedder()

// Ollama
embedder, _ := embed.NewOllamaEmbedder("http://localhost:11434", "nomic-embed-text")

// VertexAI
embedder, _ := embed.NewVertexAIEmbedder("project-id", "us-central1")

// Dummy (测试用)
embedder := embed.DummyEmbedder{}
```

0.6 模型适配器

0.6.1 文件位置

```
src/models/
```

0.6.2 Agent 接口

```go
type Agent interface {
    // 单次生成
    Generate(context.Context, string) (any, error)

    // 带文件生成
    GenerateWithFiles(context.Context, string, []File) (any, error)

    // 流式生成
```

```
    GenerateStream(context.Context, string) (<-chan StreamChunk, error)
}
```

0.6.3 支持的提供商

0.6.3.1 1. Gemini

```
// 创建 Gemini 模型
model, err := models.NewGeminiLLM(ctx, "gemini-2.5-pro", "Research summary:")

// 环境变量
export GOOGLE_API_KEY=...
// 或
export GEMINI_API_KEY=...
```

0.6.3.2 2. OpenAI

```
// 创建 OpenAI 模型
model, err := models.NewOpenAILLM(ctx, "gpt-4o-mini", "You are helpful:")

// 环境变量
export OPENAI_API_KEY=...
```

0.6.3.3 3. Anthropic

```
// 创建 Anthropic 模型
model, err := models.NewAnthropicLLM(ctx, "claude-3-sonnet-20240229", "Assistant:")

// 环境变量
export ANTHROPIC_API_KEY=...
```

0.6.3.4 4. Ollama (本地)

```
// 创建 Ollama 模型
model, err := models.NewOllamaLLM(ctx, "http://localhost:11434", "llama2", "")

// 环境变量
export OLLAMA_HOST=http://localhost:11434
```

0.6.3.5 5. Dummy (测试用)

```
// 创建 Dummy 模型（返回固定响应）
model := &models.DummyLLM{
    Response: "This is a mock response for testing.",
}
```

0.6.4 缓存模型

```
// 包装模型添加缓存功能
cachedModel := models.NewCachedLLM(model, cacheSize)

// 缓存命中时直接返回，不调用 API
```

0.7 SubAgent 系统

0.7.1 文件位置

```
src/subagents/
```

0.7.2 SubAgent 接口

```
type SubAgent interface {
    // 名称（唯一标识）
    Name() string

    // 描述（用于工具调用）
    Description() string

    // 执行任务
    Run(ctx context.Context, instruction string) (string, error)
}
```

0.7.3 预置 SubAgent

0.7.3.1 Researcher (研究员)

```
// 创建研究员
researcher := subagents.NewResearcher(model)

// 属性
Name: "researcher"
Description: "Synthesizes background information and drafts research summaries."
Persona: "You are a diligent research assistant. Provide structured findings and cite sources when
```

使用示例:

```
// 运行研究任务
result, err := researcher.Run(ctx, "Research quantum computing basics")
```

0.7.4 自定义 SubAgent

```
// 1. 定义结构体
type TranslatorSubAgent struct {
    model   models.Agent
    persona string
}

// 2. 实现接口
func (t *TranslatorSubAgent) Name() string {
    return "translator"
}

func (t *TranslatorSubAgent) Description() string {
    return "Translates text between languages"
}

func (t *TranslatorSubAgent) Run(ctx context.Context, instruction string) (string, error) {
    prompt := fmt.Sprintf("%s\n\nTask:\n%s", t.persona, instruction)
    resp, err := t.model.Generate(ctx, prompt)
    return fmt.Sprint(resp), err
}
```

```
// 3. 创建实例
translator := &TranslatorSubAgent{
    model: model,
    persona: "You are a professional translator. Translate accurately while preserving meaning and
}
```

0.7.5 SubAgent 作为工具

```
// 将 SubAgent 包装为工具
tool := agent.NewSubAgentTool(researcher)

// 注册到工具目录
catalog.Register(tool)

// 通过 UTCP 调用
result := utcpClient.InvokeTool(ctx, "researcher", map[string]any{
    "instruction": "Research quantum computing",
})
```

0.8 工具系统

0.8.1 文件位置

agent_tool.go (318 行)

0.8.2 Tool 接口

```
type Tool interface {
    // 工具规格
    Spec() ToolSpec

    // 调用工具
    Invoke(ctx context.Context, req ToolRequest) (ToolResponse, error)
}
```

0.8.3 ToolSpec

```go
type ToolSpec struct {
    Name        string         // 工具名称
    Description string         // 工具描述
    InputSchema map[string]any // JSON Schema 输入定义
}
```

0.8.4 示例：计算器工具

```go
type CalculatorTool struct{}

func (c *CalculatorTool) Spec() ToolSpec {
    return ToolSpec{
        Name: "calculator",
        Description: "Performs mathematical calculations",
        InputSchema: map[string]any{
            "type": "object",
            "properties": map[string]any{
                "expression": map[string]any{
                    "type": "string",
                    "description": "Mathematical expression to evaluate",
                },
            },
            "required": []string{"expression"},
        },
    }
}

func (c *CalculatorTool) Invoke(ctx context.Context, req ToolRequest) (ToolResponse, error) {
    expr := req.Arguments["expression"].(string)
    result := evaluateExpression(expr)
    return ToolResponse{Content: result}, nil
}
```

0.8.5 UTCP 工具调用

```go
// 1. 创建 UTCP 客户端
utcpClient := utcp.NewClient(transport)
```

```go
// 2. 注册工具提供者
provider := base.NewProvider("local_tools")
provider.RegisterTool(calculatorTool)
utcpClient.RegisterToolProvider(ctx, provider)

// 3. 调用工具
result := utcpClient.InvokeTool(ctx, "calculator", map[string]any{
    "expression": "2 + 2",
})
```

0.9 安全护栏

0.9.1 文件位置

guardrails/main.go (示例)

0.9.2 OutputGuardrails

```go
type OutputGuardrails struct {
    policies []OutputPolicy
}

type OutputPolicy interface {
    CheckOutput(ctx context.Context, output string) error
}
```

0.9.3 预置策略

0.9.3.1 1. RegexBlocklistPolicy - 正则黑名单

```go
// 创建策略
regexPolicy, err := agent.NewRegexBlocklistPolicy([]string{
    `(?i)\bpassword\s*=\s*\w+`,      // 密码
    `(?i)\bapi[_-]?key\s*=\s*\w+`,   // API 密钥
```

```
    `(?i)\bsecret\s*=\s*\w+`,        // 密钥
})
```

0.9.3.2 2. LLMEvaluatorPolicy - LLM 评估

```
// 创建评估模型
evaluatorModel := &MockEvaluatorModel{}

// 创建策略
llmPolicy := agent.NewLLMEvaluatorPolicy(evaluatorModel)

// 添加到护栏
guardrails.AddPolicy(llmPolicy)
```

0.9.4 使用示例

```
// 创建带护栏的 Agent
agent := &Agent{
    model: model,
    Guardrails: &OutputGuardrails{
        policies: []OutputPolicy{regexPolicy, llmPolicy},
    },
}

// 调用时自动检查
response, err := agent.Generate(ctx, prompt)
if err != nil {
    // 可能是安全策略拦截
    if strings.Contains(err.Error(), "safety policy violation") {
        // 处理拦截
    }
}
```

0.10 微调测试指南

0.10.1 测试 1: 修改提示词

位置: cmd/example/main.go 或 5ways_demo/main.go

```
// 修改 system prompt
flagPrefix := flag.String("prefix", "", "Optional system/prompt prefix")

// 在创建 Agent 时应用
agent := &Agent{
    systemPrompt: *flagPrefix + " You are an expert consultant.",
}
```

测试内容: - 修改角色设定（助手 → 专家顾问）- 调整输出风格（简洁 详细）- 添加领域知识（医疗、法律、金融）

0.10.2 测试 2: 修改记忆配置

位置: src/memory/engine/engine.go

```
// 调整评分权重
memOpts := &memory.Options{
    Weights: memory.ScoreWeights{
        RecencyWeight:    0.5,  // 增加新近度权重
        ImportanceWeight: 0.3,  // 减少重要性权重
        RelevanceWeight:  0.2,
    },
}

// 调整检索数量
sessionMem := memory.NewSessionMemory(bank, 32)  // 从 16 改为 32
```

测试内容: - 调整 top-k (5, 10, 20, 50) - 修改权重配比 - 测试遗忘曲线

0.10.3 测试 3: 添加 SubAgent

位置: src/subagents/ 或示例项目中

```go
// 创建翻译 SubAgent
type TranslatorSubAgent struct {
    model   models.Agent
    persona string
}

func NewTranslator(model models.Agent) *TranslatorSubAgent {
    return &TranslatorSubAgent{
        model: model,
        persona: "You are a professional translator.",
    }
}

func (t *TranslatorSubAgent) Name() string { return "translator" }
func (t *TranslatorSubAgent) Description() string { return "Translates text" }
func (t *TranslatorSubAgent) Run(ctx context.Context, instruction string) (string, error) {
    prompt := fmt.Sprintf("%s\n\nTask:\n%s", t.persona, instruction)
    resp, err := t.model.Generate(ctx, prompt)
    return fmt.Sprint(resp), err
}

// 注册到 ADK
adk.WithSubAgents(NewTranslator(model))
```

0.10.4 测试 4: 添加工具

位置: 自定义工具文件

```go
// 创建天气查询工具
type WeatherTool struct {
    apiKey string
}

func (w *WeatherTool) Spec() ToolSpec {
    return ToolSpec{
```

```
        Name: "weather",
        Description: "Get current weather for a location",
        InputSchema: map[string]any{
            "type": "object",
            "properties": map[string]any{
                "location": map[string]any{
                    "type": "string",
                    "description": "City name",
                },
            },
        },
    }
}

func (w *WeatherTool) Invoke(ctx context.Context, req ToolRequest) (ToolResponse, error) {
    location := req.Arguments["location"].(string)
    // 调用天气 API
    weather := getWeather(location, w.apiKey)
    return ToolResponse{Content: weather}, nil
}
```

0.10.5 测试 5: 修改安全护栏

位置: guardrails/main.go

```
// 添加敏感信息检测
regexPolicy, err := agent.NewRegexBlocklistPolicy([]string{
    `(?i)\bpassword\s*=\s*\w+`,
    `(?i)\bapi[_-]?key\s*=\s*\w+`,
    `\b\d{4}[- ]?\d{4}[- ]?\d{4}[- ]?\d{4}\b`,  // 信用卡号
    `\b\d{3}[- ]?\d{3}[- ]?\d{4}\b`,             // 电话号码
    `YOUR_CUSTOM_PATTERN`,                        // 自定义规则
})
```

0.11 附录

0.11.1 环境变量参考

```
# LLM API Keys
export GOOGLE_API_KEY=...       # Gemini
export GEMINI_API_KEY=...       # Gemini (别名)
export OPENAI_API_KEY=...       # OpenAI
export ANTHROPIC_API_KEY=...    # Anthropic

# Ollama
export OLLAMA_HOST=http://localhost:11434

# 数据库
export DATABASE_URL="postgres://admin:admin@localhost:5432/ragdb?sslmode=disable"
export POSTGRES_USER=admin
export POSTGRES_PASSWORD=admin
export POSTGRES_DB=ragdb
export POSTGRES_HOST=localhost
export POSTGRES_PORT=5432

# Qdrant
export QDRANT_URL=http://localhost:6333
export QDRANT_COLLECTION=adk_memories

# Embedding
export ADK_EMBED_PROVIDER=ollama
export ADK_EMBED_MODEL=nomic-embed-text
```

0.11.2 常用命令

```
# 运行示例
go run cmd/example/main.go -provider deepseek -message "Hello"

# 运行测试
go test ./...

# 运行 5ways_demo
go run cmd/example/5ways_demo/main.go
```

```
# 运行 autonomous_agent
go run cmd/example/autonomous_agent/main.go agent --message "Hello"

# 查看文档
godoc github.com/Protocol-Lattice/go-agent
```

文档结束

最后更新：2026-03-03